

File Organisation II

CEN207 Data Structures — Week 14

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	Indexes Anim 1–2 · ISAM Anim 3 · B-tree insert Anim 4
2	B-tree search/delete Anim 5–6 · B+-tree Anim 7
3	Extendible/linear hashing Anim 8–9 · external sort Anim 10–11

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.6 (design file-based storage) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
Primary/secondary index, ISAM	Sections 1–3
B-tree insert/search/delete, B+-tree	Sections 4–7
Extendible hashing, linear hashing	Sections 8–9
External merge sort, replacement selection	Sections 10–11

How the code examples work

- Every idea has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-14/c/` and `code/week-14/java/`
- Two programs create REAL files, only in a self-made lab folder

Recap — Week 13: pages, sorted files, bucket hashing

- A **page** is the unit of disk I/O — this week's currency too
- A **sorted file**: binary search over pages, $O(\log(n/B))$
- A **bucket-hashed file**: $\sim O(1)$ search, but overflow chains
- Today: structures that make growth and search both cheap

Recap — Week 4: trees and rebalancing

- A BST can degrade to $O(n)$ on adversarial insertion order
- Rebalancing: restructure locally after a change
- A B-tree rebalances whole **pages**, not single-key nodes
- Splitting and merging pages keeps height $O(\log n)$ always

Map of the week — at a glance

Indexes	Balanced trees	Dynamic hashing	External sorting
Primary, secondary, ISAM	B-tree, B+-tree	Extendible, linear	Merge sort, replacement selection

1. Primary (Sparse) Indexes

A question to start

Binary search over a file's pages already costs $O(\log(n/B))$ reads. Could a structure small enough to live in RAM cut that further?

Intuition — a book's chapter tabs

- Flipping every page to find a word: too slow
- A **tab per chapter**, holding each chapter's first word
- Check tabs (free, in your hand), open ONE chapter
- That is exactly what a sparse index does per disk page

The idea: one entry per PAGE

- Index: `(first_key, page)` pairs, one per data page
- Sorted the same way as the file itself
- Small: `1/B` the size of the data — fits in RAM, free to scan
- `find_page` : last entry whose `first_key <= key`

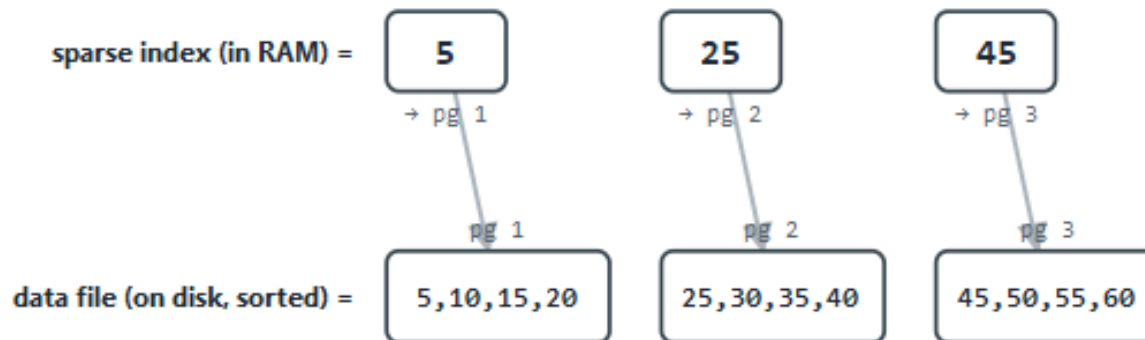
search_key: index scan, then ONE page read

- `find_page` returns `-1` if key is below every `first_key`
- `-1` means: **no disk access needed at all**
- Otherwise: read that one page, scan it for the key
- Total real disk cost: at most **one page read**

Primary index, step by step

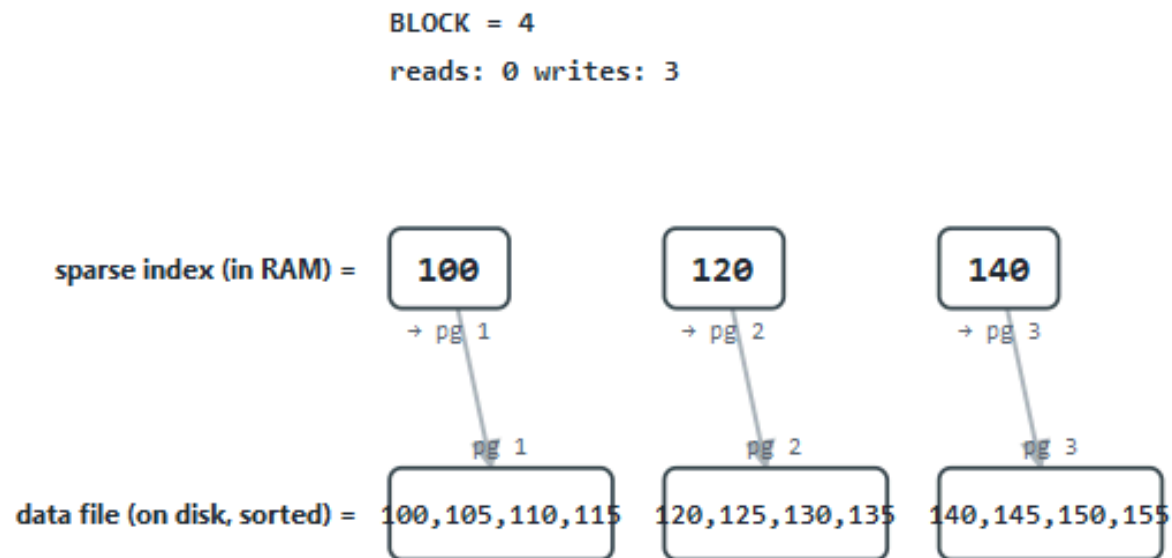
BLOCK = 4

reads: 2 writes: 3



Done: 3 searches, 2 disk reads total (writes: 3). Every search reads at most one page: thanks to the sparse index, $O(n/B)$ index scanning (in memory, free) + $O(B)$ inside the page -- total disk cost only $O(1)$ page.

Edge case — every query below range



Done: 3 searches, 0 disk reads total (writes: 3). Every search reads at most one page: thanks to the sparse index, $O(n/B)$ index scanning (in memory, free) + $O(B)$ inside the page -- total disk cost only $O(1)$ page.

Code — find_page

```
int find_page(IndexEntry index[], int idx_n, int key) {  
    int page = -1;  
    for (int i = 0; i < idx_n; i++) {  
        if (index[i].first_key <= key)  
            page = index[i].page;  
        else  
            break;  
    }  
    return page;  
}
```

Code — search_key

```
bool search_key(int data[][MAX_BLOCK], const int page_len[],
                IndexEntry index[], int idx_n, int key, int *out_page) {
    int page = find_page(index, idx_n, key);
    if (page == -1) return false;
    for (int i = 0; i < page_len[page]; i++)
        if (data[page][i] == key) { *out_page = page; return true; }
    *out_page = page;
    return false;
}
```

Complexity

- Index scan: $O(n/B)$ comparisons, but **zero** disk I/O
- One guaranteed disk read: the single data page
- Total real I/O per search: **$O(1)$** page reads
- Index space: $O(n/B)$ — proportional to pages, not records

Common mistakes

- Assuming a sparse index works on an unsorted file
- Confusing "-1: no page qualifies" with "page scanned, no match"
- Letting the index itself grow too big to fit in RAM

Mini-quiz

Why does a sparse index need only ONE entry per PAGE, not one per record?

Answer

A page read + in-page scan already finds
any record on that page. More index
entries per page would save zero extra reads.

2. Secondary (Dense) Indexes

A question to start

A primary index works on the file's own sort key. Can an index also help search on a DIFFERENT attribute, one that repeats?

Intuition — a library's subject catalogue

- Books are shelved by call number (primary order)
- A subject catalogue card exists **per book**, sorted by subject
- Cards for the same subject sit next to each other
- Each card still points to the book's real shelf location

The idea: one entry per RECORD, sorted by key

- (key, slot) pairs, ONE per record — not per page
- Sorted by the secondary key itself
- Duplicates end up **adjacent** because the index is sorted
- A match streak: scan forward until the key changes

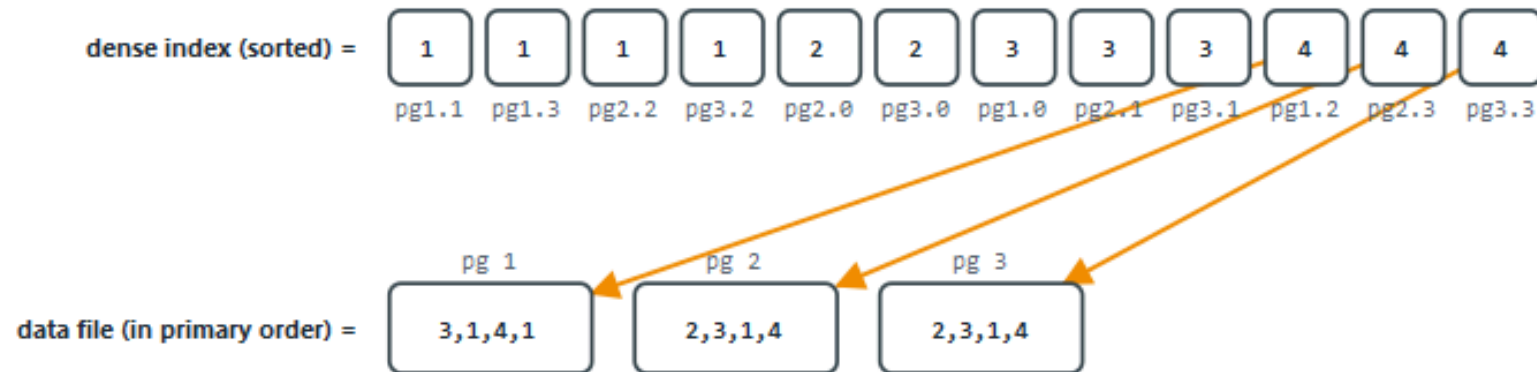
search_dense: collect a whole cluster

- Skip non-matching entries before a match starts
- Once inside a match, collect every consecutive one
- A DIFFERENT key after a match: the cluster has ended
- At most one new page read per distinct page touched

Secondary index, step by step

BLOCK = 4

reads: 6 writes: 3

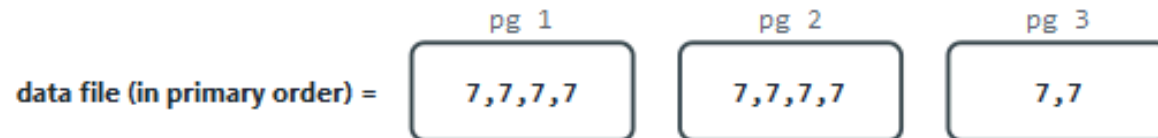
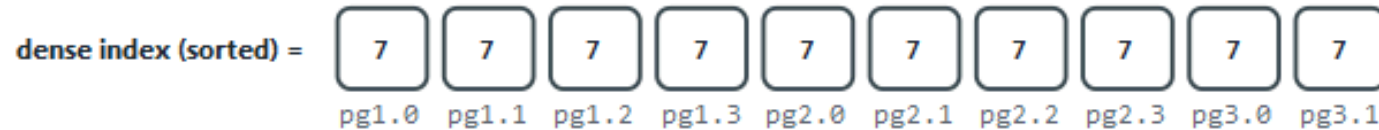


Done: 3 searches, 6 disk reads total. Because the dense index is sorted, duplicates sit next to each other: $O(n)$ index scan (in memory) + at most one new page read per match.

Edge case — all records share one key

BLOCK = 4

reads: 3 writes: 3



Done: 2 searches, 3 disk reads total. Because the dense index is sorted, duplicates sit next to each other: $O(n)$ index scan (in memory) + at most one new page read per match.

Code — search_dense

```
int search_dense(const IndexEntry index[], int n, int key,
                 int matches[], int max_matches) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (index[i].key == key) {
            if (count < max_matches) matches[count] = index[i].slot;
            count++;
        } else if (count > 0) {
            break;
        }
    }
    return count;
}
```

Complexity

- Linear scan as written: $O(n)$ worst case
- Production version: binary search to first match, $O(\log n + m)$
- Disk cost: at most one read per distinct matched page
- Space: $O(n)$ — one entry per record, not per page

Common mistakes

- Assuming the dense index is sorted by the DATA file's order
- Breaking the scan on the first non-match before a cluster starts
- Forgetting a dense index costs $O(n)$ space, not $O(n/B)$

Mini-quiz

Why must a dense index hold ONE entry per record, unlike a sparse index's one per page?

Answer

The data file is **NOT** sorted by the **secondary key**. Only one index entry per record can guarantee a correct match list.

3. ISAM: Multi-Level Index + Overflow

A question to start

Sections 1–2 assumed the file never changes. Real files grow. Where does a new record go when its page is already full?

A short history — IBM, 1960s

- **ISAM** (Indexed Sequential Access Method)
- IBM's production answer for early mainframe databases
- Used for decades in commercial data processing
- Combines a sparse index with an overflow area

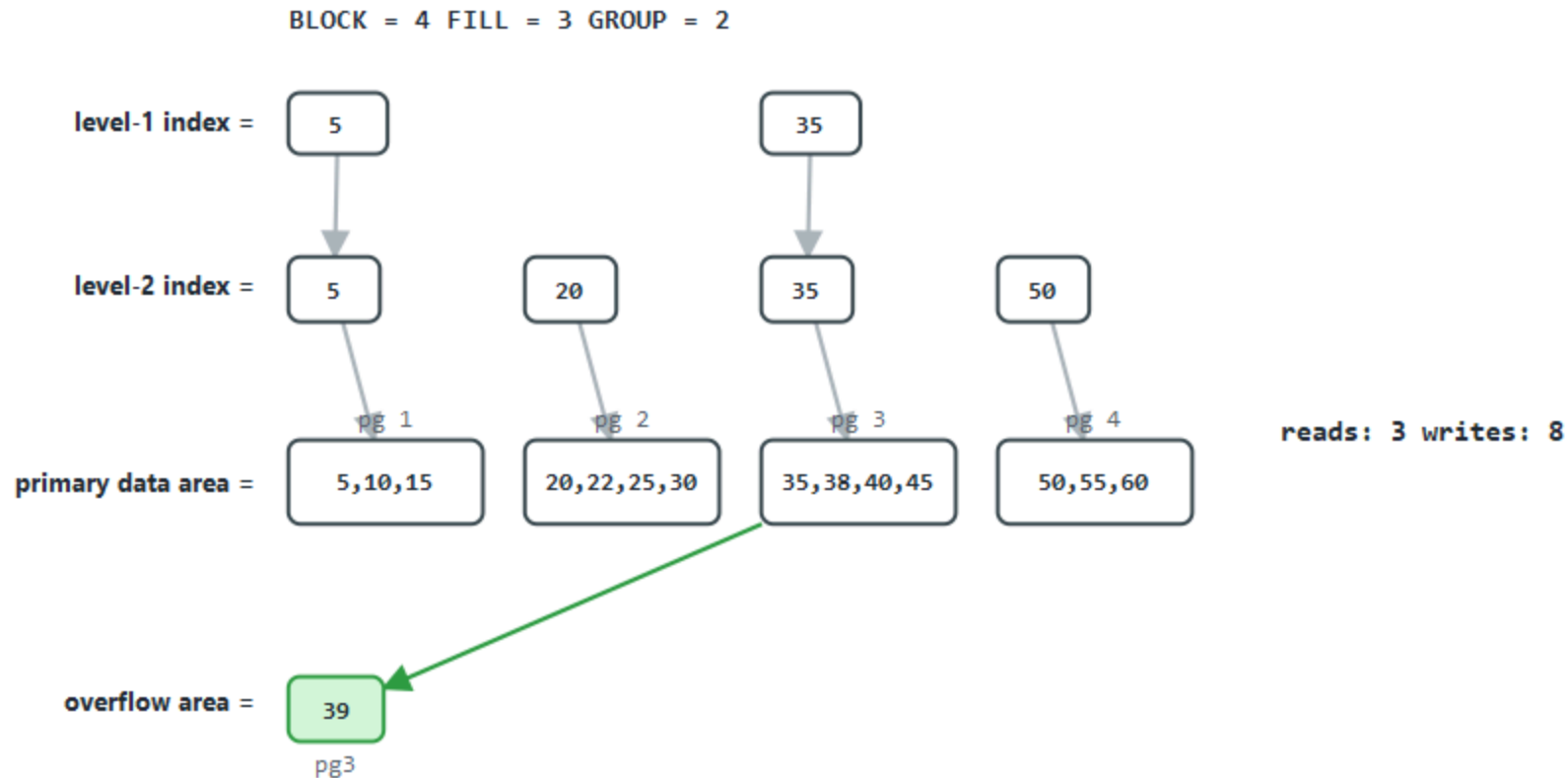
The idea: index the index, and add an escape valve

- Level-2 index: one entry per page (like Section 1)
- Level-1 index: groups every GROUP level-2 entries
- Two short scans replace one long linear scan
- A full page's new key goes to a linked OVERFLOW chain

Overflow chains: cheap now, costly later

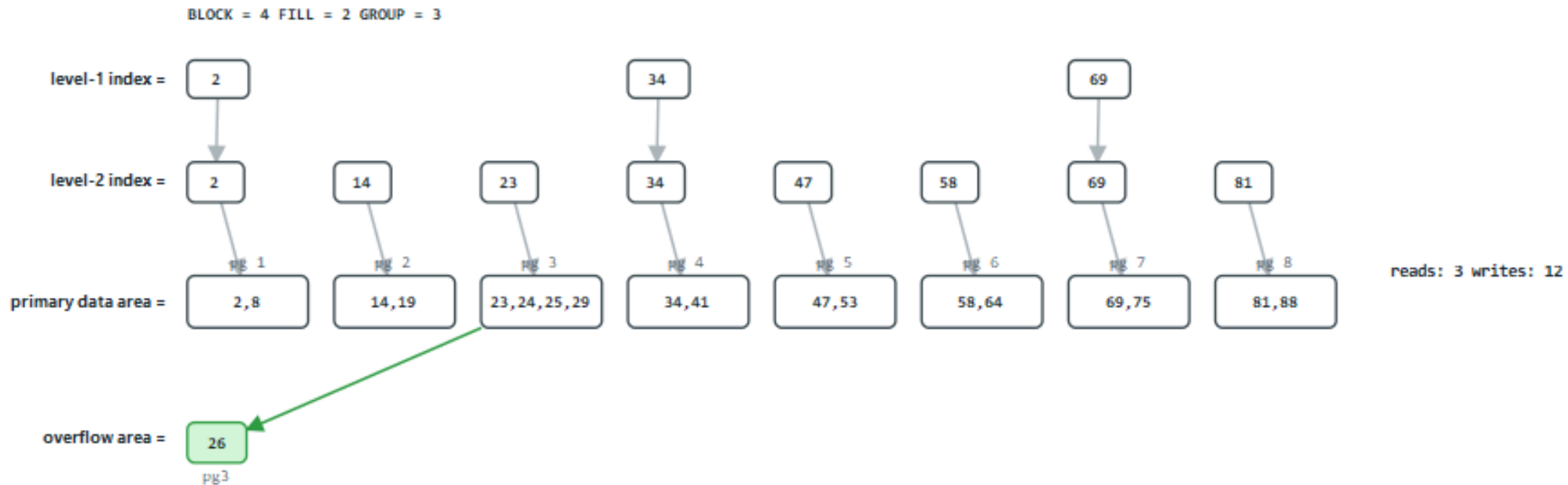
- Insert into an overflow chain: append, $O(1)$ amortised locally
- Search must walk the WHOLE chain if not on the home page
- Long chains: search performance degrades over time
- Fix: periodic **reorganisation** rebuilds the file from scratch

ISAM, step by step



Done: 3 inserts, 3 reads / 8 writes in total. As overflow chains grow, search gets slower -- which is why real ISAM files are periodically reorganized.

Edge case — chained overflow, same page



Done: 3 inserts, 3 reads / 12 writes in total. As overflow chains grow, search gets slower -- which is why real ISAM files are periodically reorganized.

Code — isam_insert

```
void isam_insert(int key) {
    int g = find_group(l1_key, l1_n, key);
    int page = find_page(l2_key, g * GROUP, group_hi(g), key);
    read_page(page);
    if (page_len[page] < BLOCK) {
        insert_sorted(page, key);
        write_page(page);
    } else {
        int walked = walk_overflow_chain(page);
        append_overflow(page, key);
    }
}
```

Complexity

- Search: $O(L)$ in-memory comparisons + 1 home-page read
- Plus one read per overflow node walked, if not found sooner
- Insert: same search + 1 write (direct) or 2 writes (overflow)
- Search degrades as chains grow — motivates reorganisation

Common mistakes

- Letting overflow chains grow unbounded, never reorganising
- Attaching a new overflow node to the wrong page's chain
- Stopping at the right GROUP without scanning its pages

Mini-quiz

Why does ISAM need MULTIPLE index levels, when Section 1's index used just one?

Answer

Even a "small" one-level index can grow too big for a huge file. Grouping under a level-1 index keeps the top scan tiny.

4. B-Trees: Insertion

A question to start

ISAM's index levels are fixed at build time. What if the index itself could grow and rebalance, with no reorganisation, ever?

A short history — Bayer and McCreight, 1972

- Rudolf Bayer, Edward McCreight, Boeing Research Labs
- "Organization and Maintenance of Large Ordered Indexes"
- "B" is read as Bayer, Boeing, or "balanced" — never settled
- Solved what ISAM's fixed structure could not: provable balance

Intuition — a filing cabinet that grows upward

- Each drawer (page) holds several folders (keys), sorted
- A full drawer splits into two half-full drawers
- The middle folder moves up to the drawer ABOVE
- The cabinet only gets a new TOP drawer when it must

Order m : how many keys per page

- Order m : at most $m-1$ keys, m children per node
- Root may hold fewer; every other node at least $\lceil m/2 \rceil - 1$
- Insert descends to a leaf, exactly like a search would
- If the leaf now holds m keys: **split**

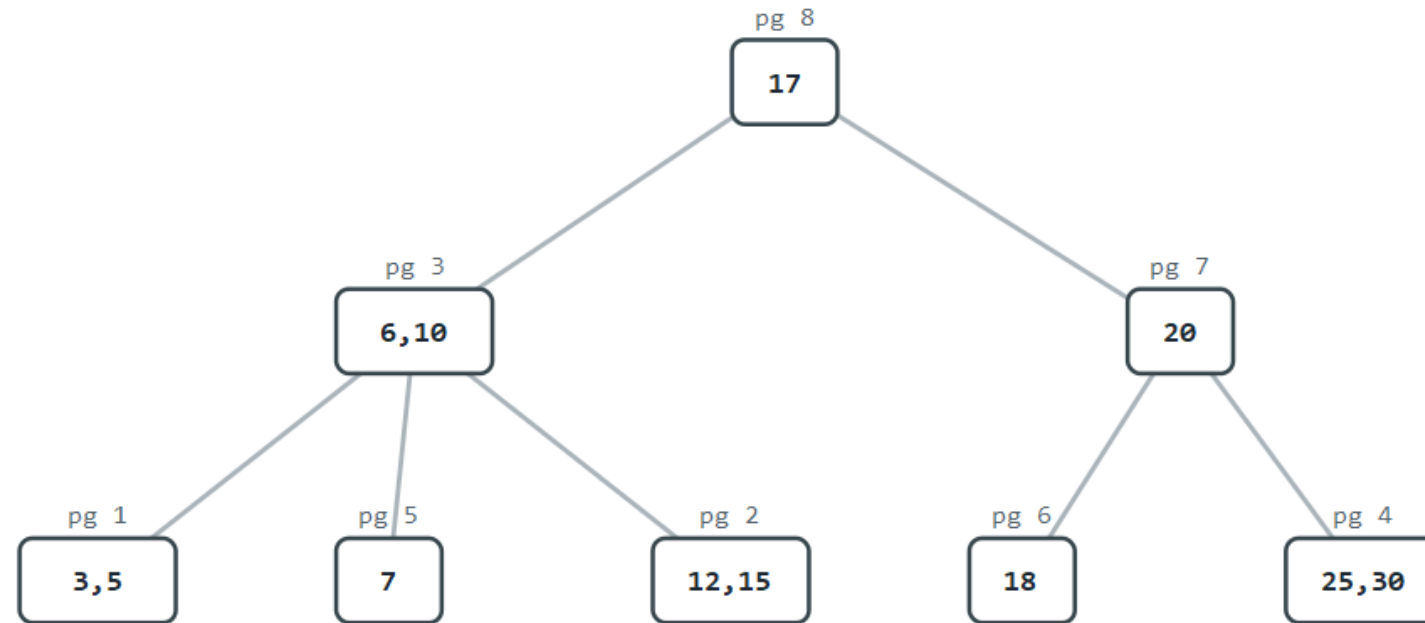
split: push the median up

- `mid = n/2` ; the median key is pushed to the parent
- Left half keeps `keys[0..mid-1]` , right half gets the rest
- If the PARENT now overflows too: split it too
- If the ROOT splits: a new root grows the tree one level

B-tree insert, step by step

ORDER = 4 (max 3 keys/node)

reads: 20 writes: 28



Done: 12 inserts, ending with 8 pages, height 2, 20 reads / 28 writes. A B-tree always stays **BALANCED**: every leaf sits at the same depth.

Edge case — order=12, never splits

ORDER = 12 (max 11 keys/node)

reads: 10 writes: 11

pg 1

3,5,7,12,16,23,29,38,41,44

Done: 10 inserts, ending with 1 pages, height 0, 10 reads / 11 writes. A B-tree always stays **BALANCED**: every leaf sits at the same depth.

Code — insert_sorted and split

```
void insert_sorted(Node *node, int key) {  
    int i = node->n - 1;  
    while (i >= 0 && node->keys[i] > key) {  
        node->keys[i + 1] = node->keys[i]; i--;  
    }  
    node->keys[i + 1] = key;  
    node->n++;  
}
```

Code — b_tree_insert

```
void b_tree_insert(BTree *t, int key) {
    Node *leaf = find_leaf(t->root, key);
    insert_sorted(leaf, key);
    Node *cur = leaf;
    while (cur->n == ORDER) {
        int median; Node *right = split(cur, &median);
        if (cur->parent == NULL) { t->root = new_root(median, cur, right); return; }
        insert_sorted(cur->parent, median);
        attach_child(cur->parent, right);
        cur = cur->parent;
    }
}
```

Complexity

- Height: $O(\log_m n)$ — order 100, 1 billion records: only 5 levels
- Insert: $O(\log_m n)$ reads to find the leaf
- Plus up to $O(\log_m n)$ splits, $O(m)$ work each
- Worst case (cascade to root): $O(m \log_m n)$

Common mistakes

- Splitting only the leaf, forgetting the parent may overflow too
- Growing the tree at the wrong end (bottom instead of top)
- Choosing an order so small ordinary data keeps splitting

Mini-quiz

Why does a B-tree's height stay $O(\log_m n)$ even under an adversarial insertion order?

Answer

Every split is local; the tree only grows taller when a split reaches the **ROOT**.

No insertion order makes a long chain.

5. B-Tree Search

A question to start

Insertion builds a balanced tree. Given one that already exists, how many pages does finding — or ruling out — one key cost?

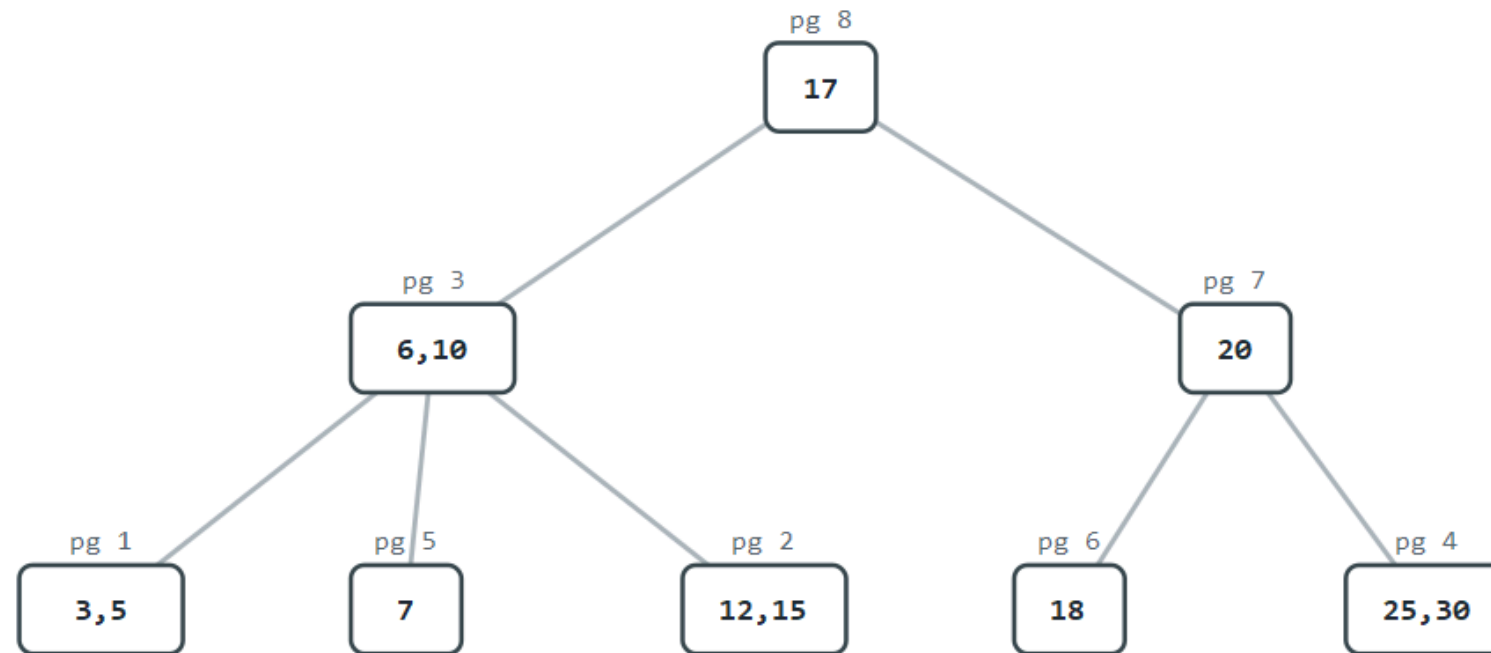
The idea: descend, comparing at every page

- Start at the root; scan its (small) sorted key list
- Exact match: done
- No match, and it's a LEAF: the key cannot exist — absent
- No match, not a leaf: descend into exactly one child

B-tree search, step by step

ORDER = 4

reads: 7



Done: 3 searches, 7 disk reads total. Every search reads at most $\text{height} + 1 = 3$ pages: $O(\log_{\text{ORDER}} n)$.

Edge case — single node, every search is 1 read

ORDER = 12

reads: 3

pg 1

3, 5, 7, 12, 16, 23, 29, 38, 41, 44

Done: 3 searches, 3 disk reads total. Every search reads at most $\text{height} + 1 = 1$ pages: $O(\log_{\text{ORDER}} n)$.

Code — b_tree_search

```
bool b_tree_search(Node *node, int key, Node **out_node) {  
    if (node == NULL) return false;  
    int i = 0;  
    while (i < node->n && key > node->keys[i]) i++;  
    if (i < node->n && key == node->keys[i]) {  
        *out_node = node; return true;  
    }  
    if (node->leaf) return false;  
    return b_tree_search(node->child[i], key, out_node);  
}
```

Complexity

- $O(\log_m n)$ page reads worst case — one per level
- Within a page: $O(\log m)$ binary, or $O(m)$ linear scan
- Same cost whether the key is found OR proven absent
- This is the direct payoff of Section 4's balanced insertion

Common mistakes

- Descending into a leaf's (uninitialised) child array
- Off-by-one in choosing which child index to descend into
- Assuming "not found" costs fewer reads than "found"

Mini-quiz

Why does B-tree search cost at most
(height + 1) reads, whichever key is searched?

Answer

Every leaf sits at exactly the same depth. A search either finds its key early, or walks to a leaf — never deeper.

6. B-Tree Deletion: Borrow and Merge

A question to start

Insertion splits an overfull page. Could deletion's mirror image — merging two too-empty pages — sometimes be avoided?

The idea: remove, then fix underflow

- Leaf key: shift-remove, simple
- Internal key: replace with its inorder PREDECESSOR
- The predecessor's own leaf is where the real delete happens
- If that leaf underflows (too few keys): fix it going up

Fixing underflow: borrow first, merge only if needed

- Left sibling has a spare key? **Borrow**: rotate through parent
- No? Try the RIGHT sibling the same way
- Neither has a spare? **Merge** with one sibling
- A merge can underflow the PARENT: the check repeats up

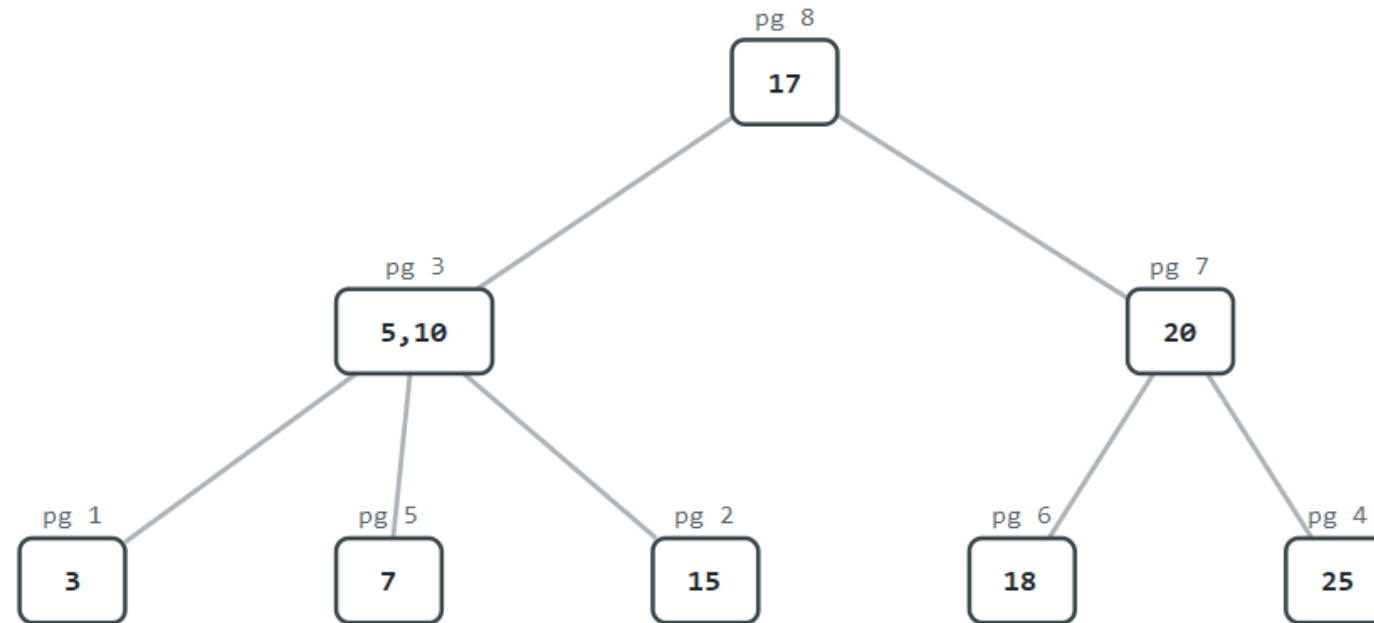
When a merge reaches the root

- A merge at the top can empty the root of all keys
- The root's one remaining child becomes the new root
- The tree **shrinks** by exactly one level
- The direct mirror of Section 4's root-split growth

B-tree delete, step by step

ORDER = 4 MIN_KEYS = 1

reads: 9 writes: 4

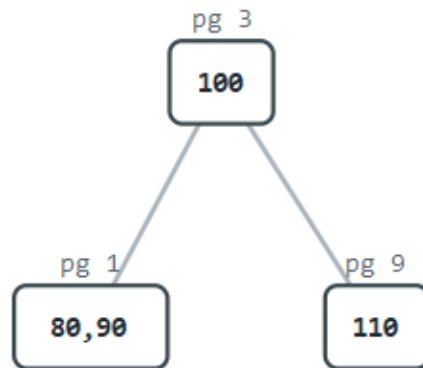


Done: 3 deletes (0 borrows, 0 merges), 9 reads / 4 writes. The tree is still **BALANCED**: every leaf at the same depth.

Edge case — delete until the root shrinks

ORDER = 3 MIN_KEYS = 1

reads: 19 writes: 19



Done: 7 deletes (0 borrows, 6 merges), 19 reads / 19 writes. The tree is still **BALANCED**: every leaf at the same depth.

Code — fix_underflow (borrow or merge)

```
void fix_underflow(Node *node) {
    while (node->parent != NULL && node->n < MIN_KEYS) {
        Node *parent = node->parent;
        int idx = child_index(parent, node);
        Node *left = idx > 0 ? parent->child[idx - 1] : NULL;
        Node *right = idx < parent->n ? parent->child[idx + 1] : NULL;
        if (left && left->n > MIN_KEYS) { borrow_from_left(node, parent, left, idx); return; }
        if (right && right->n > MIN_KEYS) { borrow_from_right(node, parent, right, idx); return; }
        if (left) { merge(left, parent, node, idx - 1); node = parent; }
        else { merge(node, parent, right, idx); node = parent; }
    }
}
```

Complexity

- Find the key: $O(\log_m n)$, same as search
- Fix-up: up to $O(\log_m n)$ merges, one level at a time
- Each merge/borrow: $O(m)$ work to shift keys and children
- Borrow: touches 3 pages; merge: removes a page + a key

Common mistakes

- Borrowing from a sibling with exactly MIN_KEYS (still underflows)
- Forgetting to move a child pointer along with a borrowed key
- Deleting the wrong copy after an internal-node replacement

Mini-quiz

Why does a B-tree prefer BORROWING over MERGING, whenever a sibling can spare a key?

Answer

Borrow touches 3 pages and resolves immediately. Merge removes a page from the parent, which can cascade further up.

7. B+-Trees: Leaf Chains and Range Queries

A question to start

"Every order in the last 30 days" is a RANGE query. A plain B-tree keeps re-climbing to the root. Can that be avoided?

The idea: keys only in leaves, leaves form a chain

- Internal nodes hold ROUTING copies only — never real data
- Every real key lives in a LEAF
- Every leaf holds a `next` pointer to the leaf on its right
- A range query: descend ONCE, then follow the chain

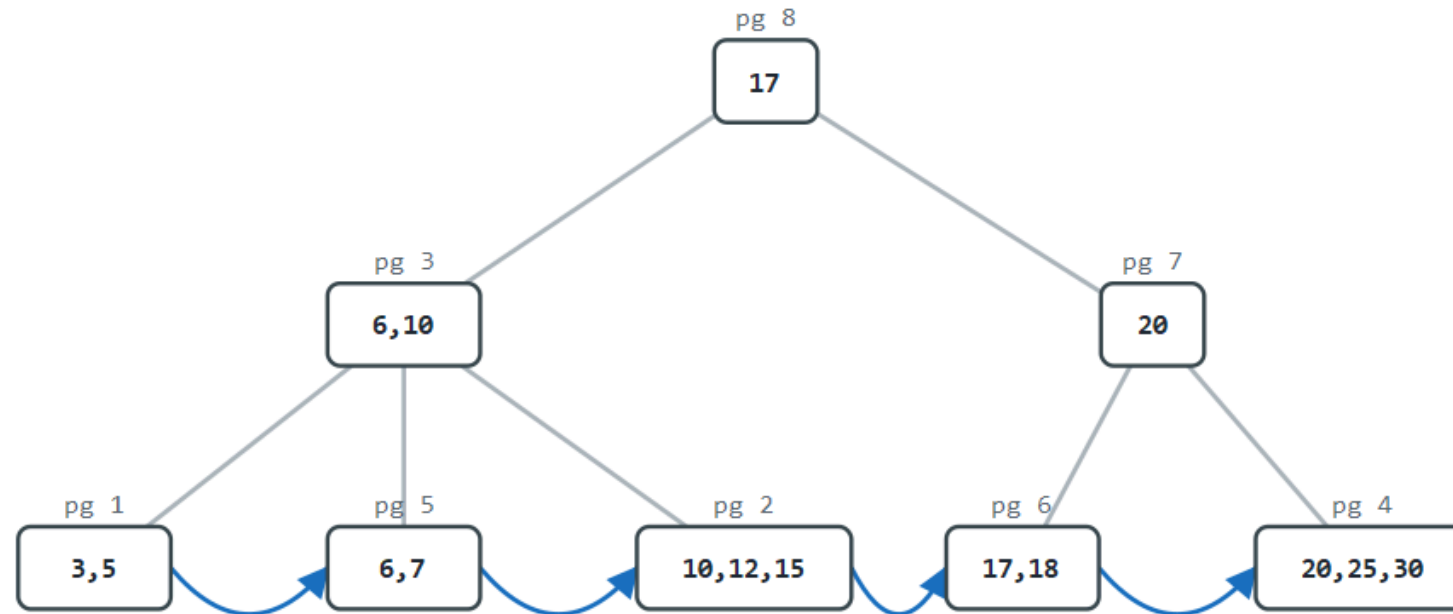
Leaf split copies; internal split removes

- Leaf splits: median is COPIED up (stays in the right leaf too)
- Internal splits: median is REMOVED (pure routing, no data lost)
- This ONE difference from Section 4 is the whole B+-tree idea
- Splicing `next` correctly is the one new bookkeeping step

B+-tree, step by step

ORDER = 4

reads: 13

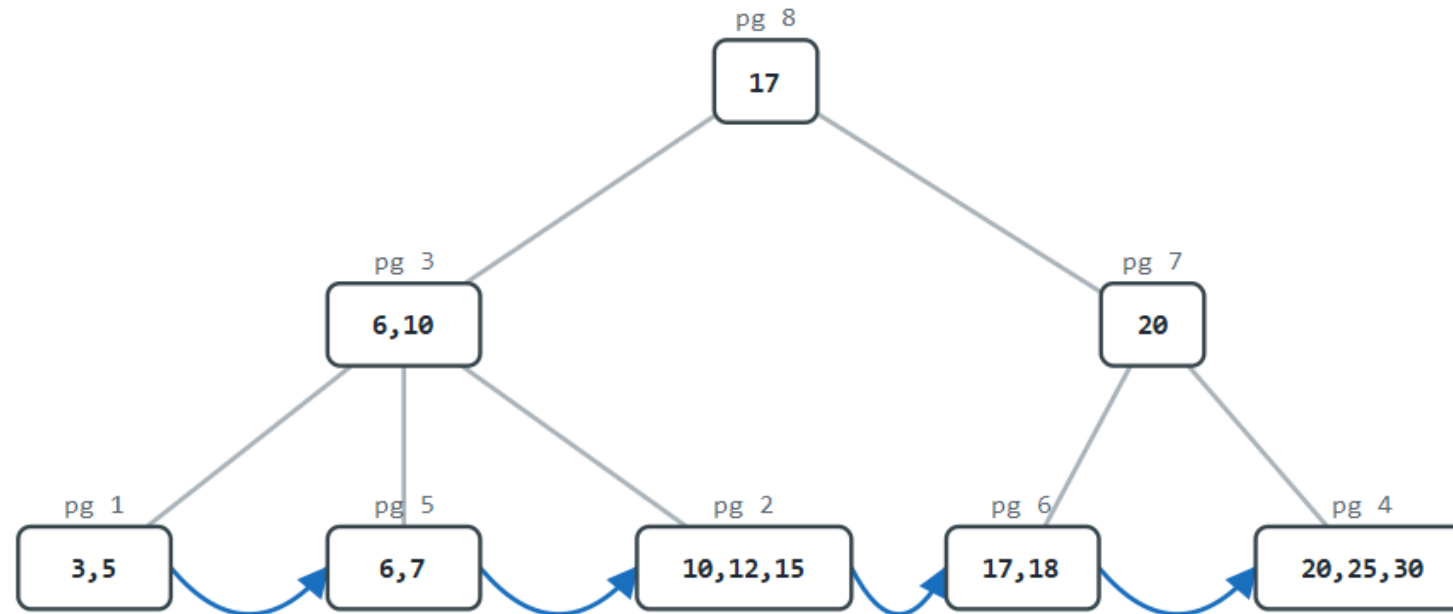


Done: 3 range queries, 13 reads total. The B+-tree payoff: descend once, then just walk the chain -- no re-descending from the root for every leaf.

Edge case — a range covering every key

ORDER = 4

reads: 7



Done: 1 range queries, 7 reads total. The B+-tree payoff: descend once, then just walk the chain -- no re-descending from the root for every leaf.

Code — range_query

```
void range_query(Node *root, int lo, int hi, int out[], int *count) {
    Node *leaf = find_leaf(root, lo);
    *count = 0;
    while (leaf != NULL) {
        for (int i = 0; i < leaf->n; i++)
            if (leaf->keys[i] >= lo && leaf->keys[i] <= hi)
                out[(*count)++] = leaf->keys[i];
        if (leaf->n > 0 && leaf->keys[leaf->n - 1] > hi) break;
        leaf = leaf->next;
    }
}
```

Complexity

- One lookup: $O(\log_m n)$, same as a plain B-tree
- Range of k matches: $O(\log_m n) + O(k/B)$ chain reads
- A plain B-tree: could cost $O(k)$ separate root descents
- The chain is the entire source of this speed-up

Common mistakes

- Copying the median on an INTERNAL split (only leaves copy)
- Forgetting to splice `next` before overwriting `leaf->next`
- Running a range query as repeated single-key searches

Mini-quiz

Why can a B+-tree's range query avoid climbing back to the root, unlike a plain B-tree's?

Answer

Every leaf knows its own next leaf, directly, $O(1)$. A plain B-tree has no such shortcut between neighbouring leaves.

8. Extendible Hashing

A question to start

Week 13's bucket-hashed file froze its bucket count at creation. Can a hashed file grow one bucket at a time, on demand?

A short history — Fagin et al., 1979

- Ronald Fagin, Jürg Nievergelt, Nicholas Pippenger, H. R. Strong
- "Extendible Hashing — A Fast Access Method for Dynamic Files"
- Key idea: directory (memory) separate from buckets (disk)
- Growth touches the cheap directory far more than disk pages

The idea: a directory, doubled only when needed

- Directory: $2^{\text{global_depth}}$ pointers, indexed by last bits
- Multiple entries can point to the SAME bucket
- Each bucket tracks its own `local_depth`
- Overflow, `local_depth == global_depth`: directory DOUBLES first

Split, then retry

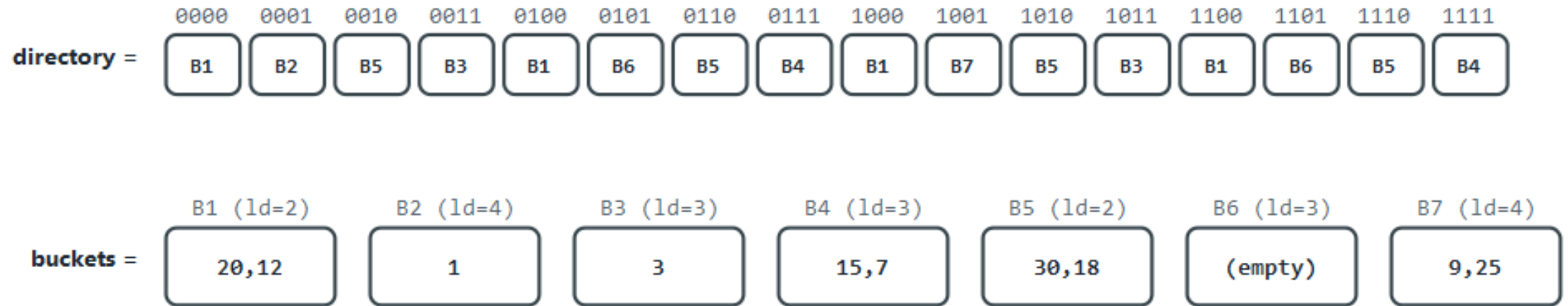
- Bucket splits: `local_depth++`, a new bucket is created
- Keys redistribute by the ONE new bit the deeper split examines
- Retry the insert: one split is not always enough
- Adversarial data can need SEVERAL splits in a row

Extendible hashing, step by step

CAPACITY = 2

reads: 16 writes: 23

global depth = 4



Done: 10 inserts, global_depth=4 (directory size 16), 7 buckets, 16 reads / 23 writes. The directory lives in memory: every insert costs at most 2 disk accesses (read + write, +1 more bucket on a split).

Edge case — cascading splits



Done: 10 inserts, global_depth=7 (directory size 128), 10 buckets, 19 reads / 29 writes. The directory lives in memory: every insert costs at most 2 disk accesses (read + write, +1 more bucket on a split).

Code — insert_key

```
void insert_key(Hash *h, int key) {
    int idx = last_bits(key, h->global_depth);
    Bucket *b = h->dir[idx];
    if (b->n < CAPACITY) { b->keys[b->n++] = key; return; }
    if (b->local_depth == h->global_depth) {
        h->global_depth++;
        double_directory(h);
    }
    split_bucket(h, b);
    insert_key(h, key);
}
```

Complexity

- $O(1)$ directory lookup (memory, free)
- 1 page read + 1 write, +2 more per split
- Splits are $O(1)$ amortised — like dynamic array doubling
- Directory space: $O(2^{\text{global_depth}})$, small unless skewed

Common mistakes

- Splitting the bucket before checking if the directory must double
- Forgetting the retry — one split does not always separate keys
- Assuming every bucket a split creates ends up non-empty

Mini-quiz

Why must the directory double BEFORE
a bucket splits, not after?

Answer

A split needs spare, more-specific slots to redirect. Without doubling first, none exist yet at the current depth.

9. Linear Hashing

A question to start

Extendible hashing needs a whole extra directory structure. Can a file grow one bucket at a time with NO directory at all?

A short history — Witold Litwin, 1980

- "Linear Hashing: A New Tool for File and Table Addressing"
- Buckets split in FIXED, round-robin order: 0, then 1, then 2...
- Completely independent of which bucket actually overflowed
- That predictability is what removes the need for a directory

The idea: a counter n tracks the next split

- $level$: how many full rounds of doubling completed
- n : how many buckets have split THIS round
- Address: $key \bmod (N_0 * 2^{level})$
- If that address $< n$ (already split): use the next level

ANY overflow splits bucket n — not the one that overflowed

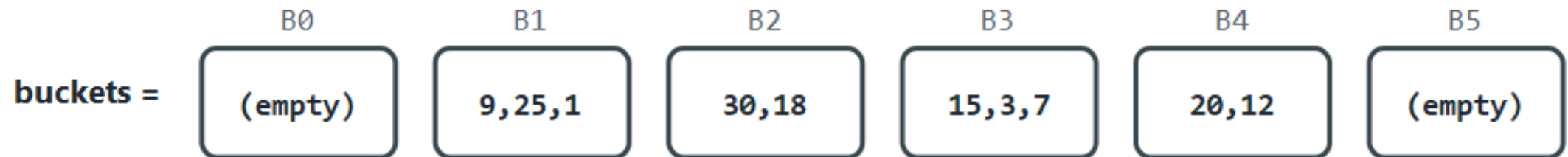
- Insert always fits: an overflowing bucket just grows
- Any overflow, anywhere, triggers splitting bucket n
- n advances; a full round resets $n=0$, $level++$
- A bucket can temporarily hold MORE than CAPACITY

Linear hashing, step by step

N0 = 4 CAPACITY = 2

reads: 10 writes: 18

level = 0 next bucket to split n = 2



Done: 10 inserts, 6 buckets (level=0, n=2), 10 reads / 18 writes. No directory was ever needed -- just a modulo and one counter (n).

Edge case — frequent splits, a full round

`N0 = 2 CAPACITY = 1`
`reads: 10 writes: 26`

level = 2 next bucket to split n = 1



Done: 10 inserts, 9 buckets (level=2, n=1), 10 reads / 26 writes. No directory was ever needed -- just a modulo and one counter (n).

Code — address and split

```
int address(int key, int level, int n) {
    int a = key % (N0 << level);
    if (a < n) a = key % (N0 << (level + 1));
    return a;
}

void split(Hash *h) {
    int new_index = (N0 << h->level) + h->n;
    rehash_into(h, h->n, new_index);
    h->n++;
    if (h->n == (N0 << h->level)) { h->n = 0; h->level++; }
}
```

Complexity

- $O(1)$ average insert — same as extendible hashing
- ZERO directory memory: only two integers, level and n
- Trade-off: an individual bucket's size is less tightly bounded
- An overflowing-but-not-yet-due bucket just keeps growing

Common mistakes

- Splitting the OVERFLOWING bucket instead of bucket n
- Forgetting the `a < n` bump when computing an address
- Never resetting n to 0 and advancing level

Mini-quiz

Why can linear hashing avoid a directory entirely, when extendible hashing cannot?

Answer

The split target is always predictable from level and n alone. No need to record which entries point to which bucket.

10. External Merge Sort

A question to start

Week 10 sorted arrays that fit in RAM.
A billion-record file does not. What
changes when "the other half" lives on disk?

A short history — the tape sorting era

- 1950s–60s mainframes sorted files far larger than RAM
- Magnetic tape: sequential access only, no random seeks
- "Merge" was often the ONLY efficient operation available
- Knuth's TAOCP Vol. 3 (1973): the classic exhaustive reference

The idea: small sorted runs, then a k-way merge

- Phase 1: chunks of RUN_SIZE, sorted in RAM, written as RUNS
- Phase 2: merge up to FAN_IN runs, ONE buffer per run
- Pick the smallest current buffer value, write it, refill
- Repeat passes until exactly ONE run remains

A lone leftover run just carries forward

- If a group has only 1 run left, nothing to merge with
- Carry it forward to the next pass untouched — zero I/O
- Wasting a read+write on a solo run is a common mistake
- This optimisation matters most with an odd number of runs

External merge sort, step by step

`RUN_SIZE = 4 FAN_IN = 2`

`reads: 7 writes: 5`

unsorted input =

40,11,27,8,33,2,45,16,38,23,5,30

pass 0 (runs) =

8,11,27,40

2,16,33,45

5,23,30,38

pass 1

2,8,11,16,27,33,40,45

5,23,30,38

pass 2

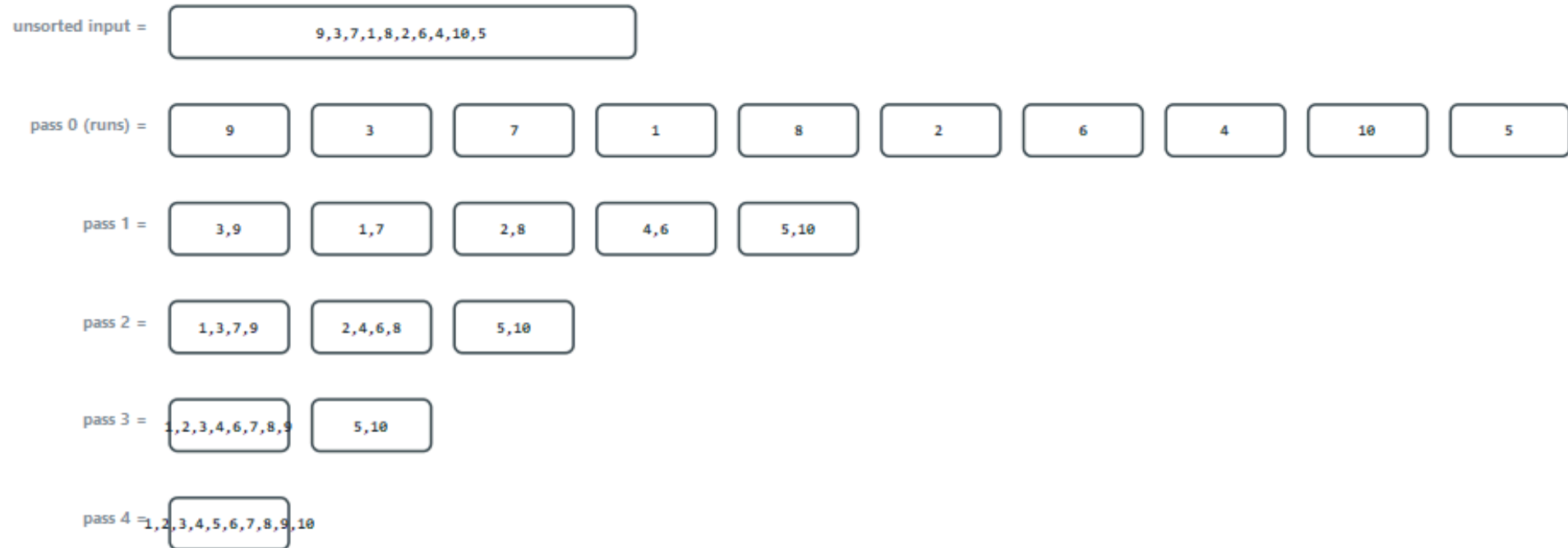
2,5,8,11,16,23,27,30,33,38,40,45

Done: 2 merge pass(es), 7 reads / 5 writes. The result is one sorted run: [2,5,8,11,16,23,27,30,33,38,40,45].
However many runs a pass scans, RAM only ever holds FAN_IN buffers.

Edge case — RUN_SIZE=1, many passes

RUN_SIZE = 1 FAN_IN = 2

reads: 28 writes: 19



Done: 4 merge pass(es), 28 reads / 19 writes. The result is one sorted run: [1,2,3,4,5,6,7,8,9,10]. However many runs a pass scans, RAM only ever holds FAN_IN buffers.

Code — merge_group

```
Run merge_group(Run group[], int g) {  
    int ptr[FAN_IN] = {0};  
    Run out = new_run();  
    while (1) {  
        int best = -1, best_val = INT_MAX;  
        for (int i = 0; i < g; i++)  
            if (ptr[i] < group[i].n && group[i].keys[ptr[i]] < best_val)  
                { best_val = group[i].keys[ptr[i]]; best = i; }  
        if (best == -1) break;  
        append(&out, best_val);  
        ptr[best]++;  
    }  
    return out;  
}
```

This program creates REAL files — safely

- Real run files, but only in a self-made LAB FOLDER
- `lab_external_merge_sort/`, created by the program itself
- Every file, and the folder, removed before the program exits
- Nothing is ever written anywhere else on disk

Complexity

- Phase 1: $\text{ceil}(n/\text{RUN_SIZE})$ runs, 1 read + 1 write each
- Passes: $O(\log_{\text{FAN_IN}}(n/\text{RUN_SIZE}))$
- Each pass touches every record once: $O(n/B)$ I/O per pass
- Total: $O((n/B) \cdot \log_{\text{FAN_IN}}(n/\text{RUN_SIZE}))$

Common mistakes

- Merging a lone leftover run anyway (wasted read + write)
- Buffering an entire run in RAM during a merge
- Choosing FAN_IN larger than RAM can actually hold

Mini-quiz

Why does external merge sort need only $O(\text{FAN_IN})$ RAM buffers, however huge the file?

Answer

A k-way merge only needs each run's **CURRENT front value**. The runs are already sorted — nothing more is needed.

11. Replacement Selection: Longer Runs

A question to start

Section 10's Phase 1 always makes runs of exactly RUN_SIZE. Can the same RAM produce runs LONGER than RUN_SIZE?

The idea: current vs. next, decided by the last WRITE

- Keep a small RAM window (a min-heap in practice)
- Extract the smallest CURRENT-tagged record, write it
- New record `>= last written` ? Tag CURRENT (can extend the run)
- New record `< last written` ? Tag NEXT (waits for the next run)

Best case, average case, worst case

- All window items NEXT-tagged: current run ends, retag, new run
- Random data: runs average about **2x** the window size m
- Already-sorted input: ONE single run, however large the file
- Strictly descending input: worst case, exactly m per run

Replacement selection, step by step

RAM_SIZE = 4

reads: 12 writes: 12

input (to be read in order) =

40, 11, 27, 8, 33, 2, 45, 16, 38, 23, 5, 30

RAM window =

output (current run) =

run 1

8, 11, 27, 33, 40, 45

run 2

2, 5, 16, 23, 30, 38

Done: 2 run(s) (lengths: 6, 6), 12 reads / 12 writes. Even though RAM_SIZE=4, runs usually come out much longer than 4 -- about 2x on average for random data.

Edge case — best case, $\text{RAM} \geq n$

RAM_SIZE = 15

reads: 10 writes: 10

input (to be read in order) =

9,3,7,1,8,2,6,4,10,5

RAM window =

run 1

output (current run) =

1,2,3,4,5,6,7,8,9,10

Done: 1 run(s) (lengths: 10), 10 reads / 10 writes. Even though $\text{RAM_SIZE}=15$, runs usually come out much longer than 15 -- about 2x on average for random data.

Code — extract_min_current

```
int extract_min_current(Item window[], int w) {  
    int best = -1;  
    for (int i = 0; i < w; i++)  
        if (window[i].tag == CURRENT &&  
            (best == -1 || window[i].val < window[best].val))  
            best = i;  
    return best;  
}
```

This program also creates REAL files — safely

- `lab_replacement_selection/`, created by the program itself
- Every run file removed before the program exits
- Run boundaries match this week's animation exactly
- A production version keeps the window as a real min-heap

Complexity

- $O(n)$ total I/O for run creation — same as Section 10 Phase 1
- $E[\text{run length}] \approx 2m$ on random data — half as many later passes
- `extract_min_current`: $O(m)$ here, $O(\log m)$ with a real heap
- Worst case (descending input): no better than plain chunking

Common mistakes

- Comparing against the window's minimum, not the last WRITE
- Forgetting to reset last_written when a new run starts
- Assuming the "2x" average also holds in the worst case

Mini-quiz

Why does replacement selection produce runs about TWICE the window size m , on average?

Answer

Roughly half the window stays **CURRENT**
at any moment, continuously refreshed.
The run grows to about 2m before ending.

12. Choosing a File Organisation

Comparison table (1/2)

Structure	Search	Insert
Sorted file (Wk 13)	$O(\log(n/B))$	$O(n/B)$ shifts
Bucket hash (Wk 13)	$\sim O(1)$ + overflow	$\sim O(1)$ + overflow
Primary/secondary index	$O(1)$ page + free scan	Expensive
ISAM	$O(L)$ + overflow chain	$O(L)$ + append

Comparison table (2/2)

Structure	Range query	Reorganise?
B-tree	Repeated descents	Never
B+ -tree	Excellent (chain)	Never
Extendible hashing	Poor (no order)	Never
Linear hashing	Poor (no order)	Never

The one big decision

- Do you need ORDER (range queries, sorted iteration)?
- Or only ever EXACT-MATCH lookups?
- Order matters: a B+-tree's leaf chain is very hard to beat
- Exact-match only, ever-growing file: hashing usually wins

Summary (1/2)

- **Indexes:** primary (sparse), secondary (dense), ISAM
- ISAM adds index LEVELS and an OVERFLOW AREA
- **B-trees** (Bayer/McCreight 1972): split, search, borrow/merge
- **B+ -trees:** leaf CHAIN makes range queries fast

Summary (2/2)

- **Extendible hashing** (Fagin et al. 1979): directory, doubled
- **Linear hashing** (Litwin 1980): no directory, round-robin splits
- **External merge sort**: small runs, k-way merge, few buffers
- **Replacement selection**: ~2x longer runs from the same RAM

Self-Check Quiz Recap

1. Why does a primary index cost ZERO disk I/O to scan?

It is small enough to stay resident in memory — one entry per PAGE, not per record.

2. Why must a dense index be sorted by the SECONDARY key?

The data file's own order tells you nothing about where matching records are — only the index's own sort order does.

3. What does ISAM's overflow area let a file avoid doing immediately?

Re-splitting or reorganising the whole file on every insert — at the cost of degrading search as chains grow.

4. Why does a B-tree split always push exactly ONE key up?

The median is the one key that fits cleanly into **NEITHER** half after the split — so it is promoted.

5. Why does B-tree search cost at most (height + 1) reads?

Every leaf sits at exactly the same depth — insertion only ever grows the tree from the root, never a leaf.

6. Why does a B-tree prefer borrowing over merging?

**Borrowing touches only 3 pages and resolves immediately;
merging can cascade further up the tree.**

7. What ONE structural change makes a B+-tree's range queries fast?

Every leaf holds a next pointer to its right neighbour — a chain that needs no climbing back to the root.

8. Why must extendible hashing's directory sometimes double?

Only when the overflowing bucket's `local_depth` has caught up to `global_depth` — other buckets are unaffected.

9. Why can linear hashing split a bucket OTHER than the one that overflowed?

It commits in advance to a fixed round-robin split order, independent of which bucket happens to overflow.

10. Why does replacement selection typically double the window size m ?

The window keeps refilling: roughly half stays CURRENT-tagged at any moment, letting the run grow to ~2m.

Next week

Week 15 — Final Project Demonstrations

No new algorithms this week: teams present their project's file-organisation or storage component, before Week 16's final exam period.

References (1/2)

- Course syllabus, Week 14: `CEN207-2026-2027-Guz-Izlence.en.md`
- Bayer, McCreight (1972). "Organization and Maintenance of Large Ordered Indexes"
- Fagin, Nievergelt, Pippenger, Strong (1979). "Extendible Hashing"
- Litwin (1980). "Linear Hashing: A New Tool for File and Table Addressing"

References (2/2)

- Knuth. *TAOCP, Vol. 3: Sorting and Searching*, 2nd ed
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*. MIT Press
- Sedgewick, Wayne. *Algorithms*, 4th ed. Addison-Wesley
- [williamfiset/Algorithms](#) · Programiz DSA